

5. How to Manage Eventual Consistency

Hugo Filipe Oliveira Rocha¹

(1) Ermesinde, Portugal

This chapter covers:

- Why eventual consistency is a consequence of event-driven systems and its impact on the business and consumers
- Using event schema to avoid eventual consistency
- Applying domain boundaries to manage eventual consistency
- Using event versioning as a mean to react to inconsistent information
- Applying the end-to-end principle to guarantee a consistent flow
- Appearing consistent by maintaining a small inconsistency window

The funny thing about consistency in software engineering is that the meaning is not very consistent. Puns aside, the definition of consistency varies from context to context, and there are several levels of strong and weak consistency. Strong consistency is the consistency we are usually used to and provided by the traditional ACID (atomicity, consistency, isolation, and durability) properties. There are several degrees of weak consistency¹ that offer different levels of safety. Similar to what we would configure in a transaction isolation level, strong consistency would be the serializable level, while an example of weak consistency could be read uncommitted or read committed.

When dealing with distributed systems, there are several degrees of consistency, each with its peculiarities. I won't detail each of them since it's not in the scope of this book's contents. Still, the duality between availability and strong consistency usually triggers a lively debate illustrated by the CAP theorem detailed in Section [5.1](#).

Typical monolithic applications enjoy the comfort of ACID properties and the traditional meaning of consistency. Transactional and relational databases usually treat data as a single copy; changes are atomic and instantaneous. If you change a record, you don't have to worry about any

concurrent write since the operation is atomic. No one else can modify it simultaneously, and it will be available to anyone reading it instantaneously. When you read that record, you are guaranteed to read the last possible value.

These kinds of guarantees are typically available on monolithic applications. They simplify most developments and provide an easy solution to otherwise complex problems. This meaning of consistency is the foundation for many applications and is wired into our minds and our customers' minds while using the software. Looking back, this level of consistency is a warm fuzzy blanket on a dark cold night. When moving to an event-driven architecture, you most likely will need to leave that blanket behind and embrace the cold. Don't worry, the cold is always better than the heat of a SQL database CPU running at 100%.

Interactions between components in an event-driven architecture are often with asynchronous communications. Traditionally, communications between microservices are point-to-point synchronous communications. Often microservices expose APIs so other services can use their functionality and access their data. As exemplified in Figure 5-1, these APIs usually use synchronous communication like REST, and service 1 sends a request and obtains the response immediately. When using event-driven microservices, service 1 publishes an event which is handled by service 2, but no direct communication happens between the two services, and is always mediated through the event broker; the services don't know each other.

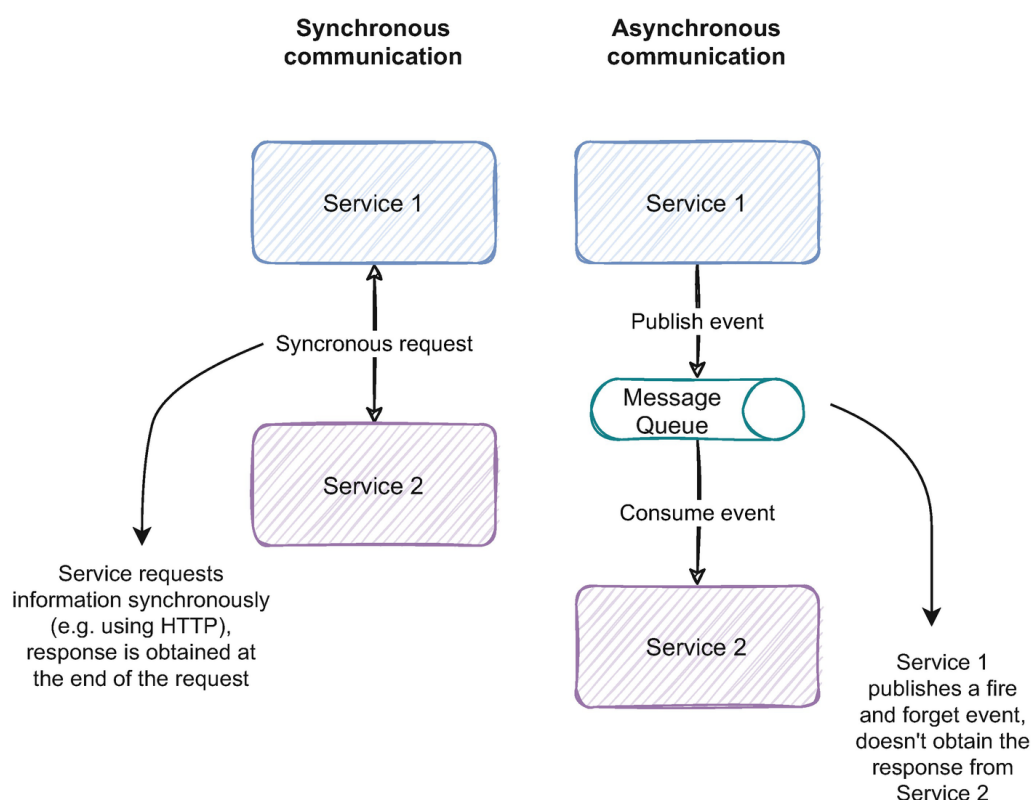


Figure 5-1 Comparison between synchronous and asynchronous microservice communication

It is often preached about the advantages of decoupling components through message queues; while this is undoubtedly useful, it also has the consequence of making the propagation of changes in the system asynchronous. The asynchronous decoupling produces what is usually named eventual consistency; reads on different parts of the system might return stale data, but given enough time, all reads will eventually return the same value. In event-driven architectures, all services processing events for a given entity will eventually converge to the same state as soon as they process those events. Eventual consistency is a specific form of weak consistency since it doesn't guarantee the stronger guarantees of ACID properties, for example. NoSQL databases have widely adopted eventual consistency as a means to enable high scalability and provide availability guarantees that are hard (or impossible) to achieve with strong consistency models in a distributed environment. Section [5.1](#) will further detail this topic and its relation to the CAP theorem.

Having components that possibly return stale values impacts the system's users and has an even higher impact on other services that depend on that information. When a service's domain logic depends on information from an eventually consistent read model, it might produce erroneous results due to the stale information. It is fundamental to know how to handle eventual consistency to leverage these impacts.

This chapter provides several techniques to deal with and leverage eventual consistency in an event-driven architecture. In Section [5.2](#), you will learn how event schema design can avoid eventual consistency. The impacts of event schema design are further detailed in Chapter [8](#).

In Section [5.3](#), you will learn to use domain boundaries to contain the impacts of eventual consistency. This relates to the choreography pattern described in Chapter [4](#). Section [5.4](#) will detail how to use event versioning to detect stale information and use compensation strategies to deal with eventual consistency.

The end-to-end principle can guarantee consistency in the end-to-end flow while allowing eventual consistency on the system's smaller components. This will be detailed in Section [5.5](#) and include a real-world example of its use. Section [5.6](#) will weigh eventual consistency with small inconsistency windows and show how real-world uses of NoSQL databases can apply it in production. You will learn how the same strategy can be applied to event-driven architectures.

5.1 The Impacts of Eventual Consistency and the Need for Alignment with the Business

In this section, you will learn the impacts of eventual consistency and understand that not every component needs or should be eventually consistent. There's no magic solution to eventual consistency, just ways to leverage it. It should be a conscious decision to adopt it and needs alignment with the system's business owners. There is also the need for a deliberate strategy to deal with your system's future consumers and should be clear to them that the service might return stale data. This section will also detail and illustrate the impact of eventual consistency and teach you to weigh the tradeoffs.

Let's illustrate eventual consistency with an example. Figure 5-2 depicts a user buying a product on an eCommerce platform based on a single application. At the moment of the purchase, the system would need to validate the available stock and create the order. The subscription service would also need to validate the remaining stock and notify interested parties if the product has just one stock unit left.

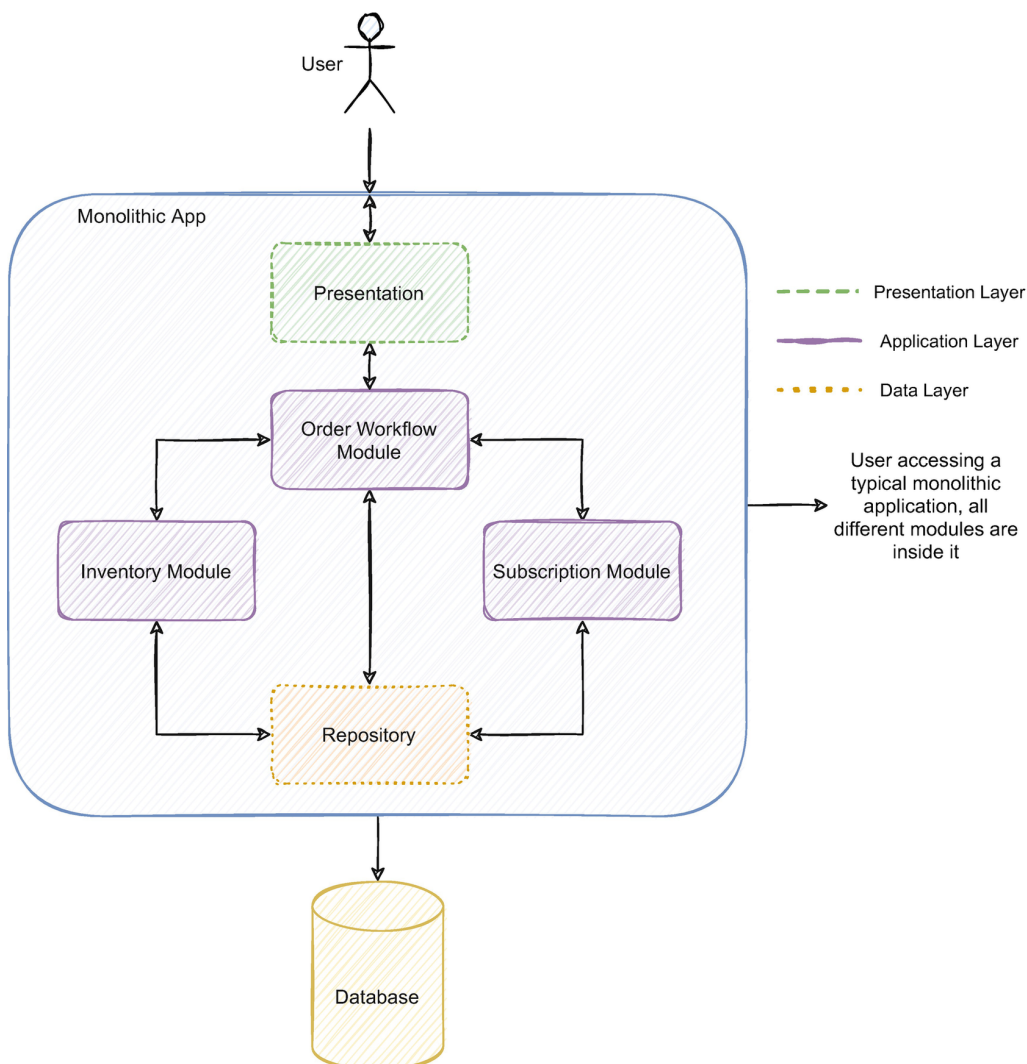


Figure 5-2 Monolithic application with several different modules

Since there is only one application, each operation is synchronous. The logic and the domain flow of the order are sequential inside that application. The database changes can occur atomically, and the information about, for example, the product's stock on the platform will be updated when the user orders the product. The new stock will be instantly visible to every user on the platform.

Now let's see the same example as before but with each module with an independent event-driven microservice as pictured in Figure 5-3. The order service would create the order and then publish an event informing about the operation. Inventory service would then handle that event and update the stock for that product. Since these two operations are asynchronous between the moment the user created the order and the moment the service updated the stock, there could be another order for the same product. Since the stock model wasn't up to date yet, the user could view and order stock for that product. The stale data and the concurrency of the orders would produce a duplicate order for stock that might not be available.

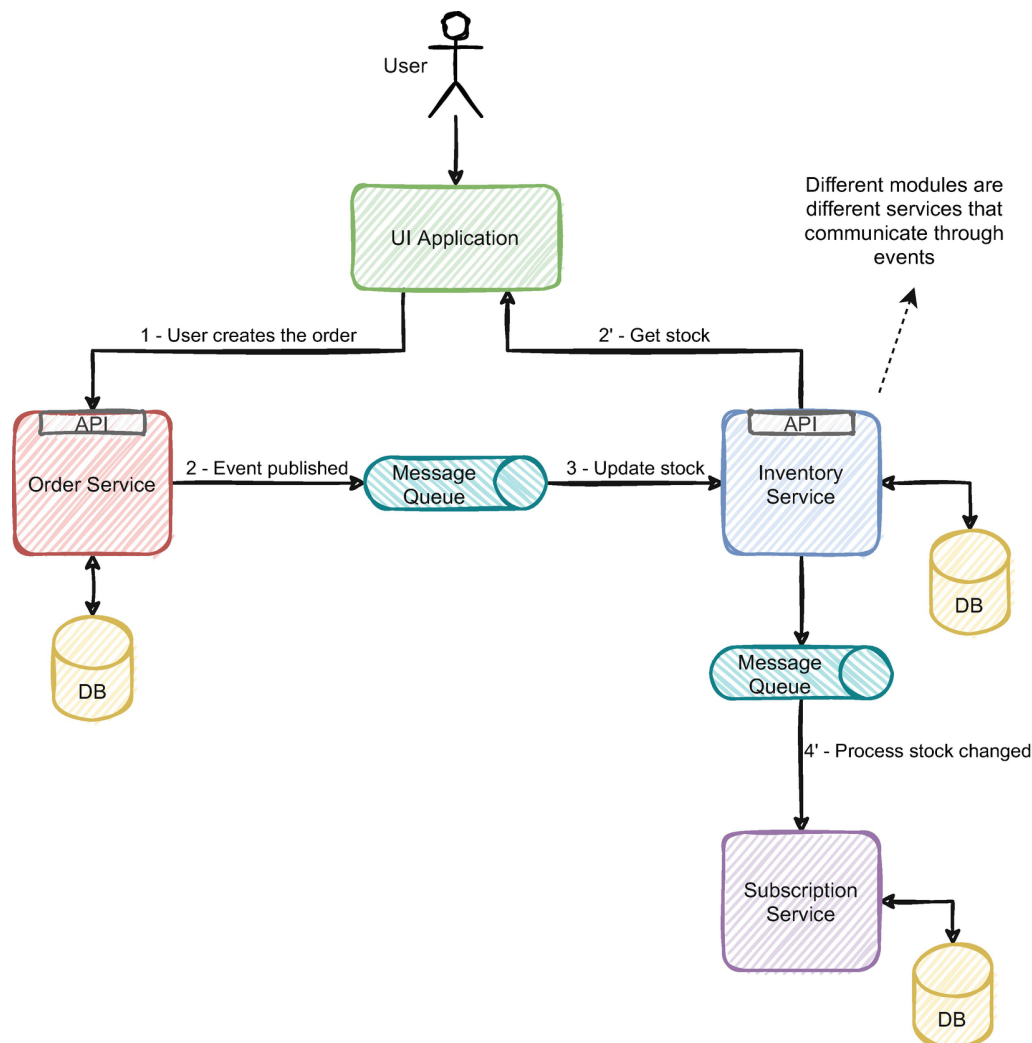


Figure 5-3 Event-driven approach to the same application

Distributed systems like event-driven architectures have two core properties²: safety and liveness. These two properties provide the foun-

dation to understand why eventual consistency is a challenge.

5.1.1 Safety

The safety property guarantees that no “bad things” happen in the system. Bad things are situations that could make the data invalid or to have a state that was never on the system (e.g., concurrency issues inside the system, possibility of deadlocks, unordered operations, etc.). Typical eventual consistency has no safety properties; however, in an event-driven system, we can avoid most of these issues. How to solve them is explored in Chapters [6](#), [7](#), and [8](#). An important consideration is while the system is inside the inconsistency window (the time between an update occurs and the change propagates throughout the system), the state returned can be invalid, which might be an issue depending on the use case. This chapter discusses possible solutions for these issues. If we model events to be small, the partial state of the entity we build through them might be temporarily invalid. For example, if you change your profile’s full address and the system processing it sends partial events for the street name and the city, the built model might have the new street name and the old city temporarily. Chapter [8](#) explores this subject in greater detail.

5.1.2 Liveliness

The liveliness property guarantees that “good things” happen in the system. In this context, the most important consequence is higher availability. Availability is one of the most useful advantages of event-driven microservices as they endow the system with higher resilience, enabling for higher percentages of the service’s availability.

The first most obvious consequence is for the users of the system. Changes by the system’s users might take time to propagate to every service in the ecosystem, impacting the tools. I’m guessing you probably used an application that after doing an update took a few refreshes of the page to see those changes reflected. Although it is a dubious user experience and mildly infuriating, it might be acceptable to live with depending on the business. There might also be the case that it is better not to be available than displaying stale information. The most crucial property in this situation is the length of the inconsistency window; if it is small enough, it won’t be noticeable, and as far as users are concerned, fast eventually consistency is strong consistency as explored in Section [5.6](#).

There are also ways to design UIs to be more compatible with asynchronous systems, as will be discussed in Chapter [9](#).

When there is business-critical functionality built on top of eventual consistency, you need to weigh whether forfeiting consistency in favor of availability is acceptable to the business. There might be use cases where not accessing the data is preferable to base decisions on stale information. We saw this firsthand on a system that managed an eCommerce platform products' visibility. The business had several agreements with brands to not allow certain categories of products to be available in specific regions. The punctual delays in the UIs due to peaks in load made the business completely stop using that feature to manage it in more error-prone manual alternatives because they simply couldn't trust the system. This example highlights the need to understand the functionality's criticality and to weigh if another solution is preferable.

The issue also gains a different magnitude in synchronous HTTP calls between services. The example in Figure [5-3](#) illustrates this issue; the UI application does an HTTP call to the inventory service API to obtain the stock after creating the order to validate if there is just only one stock unit left to notify customers. The stock returned by the inventory service might not be up to date since both the subscription service and inventory service are processing it. The stale data might result in two possible faults if the subscription service processes the order created event faster than the inventory service:

- The stock quantity is two, resulting in no notification sent since it wasn't the last stock unit.
- The stock quantity is one, resulting in the service sending a repeated notification for a stock quantity no longer available.

When a service has domain logic that depends on synchronous data from a different service, this issue will always have a chance to happen even if the inconsistency window is small. A peak in the service load might cause it to lag, producing stale responses for every dependent service.

In this case, living with eventual consistency won't be enough, and we require more deliberate strategies that are discussed further in the next chapters.

5.1.3 The CAP Theorem in the Real World

This subsection will detail further the impacts of eventual consistency and how they relate to the CAP theorem. Eventual consistency has been highly debated in the context of the CAP theorem as a means to justify its presence on event-driven architectures. By learning its meaning and real-world impacts on real production databases that deal with the same challenges, you will be able to understand the tradeoffs between availability and consistency, as pictured in Figure 5-4.

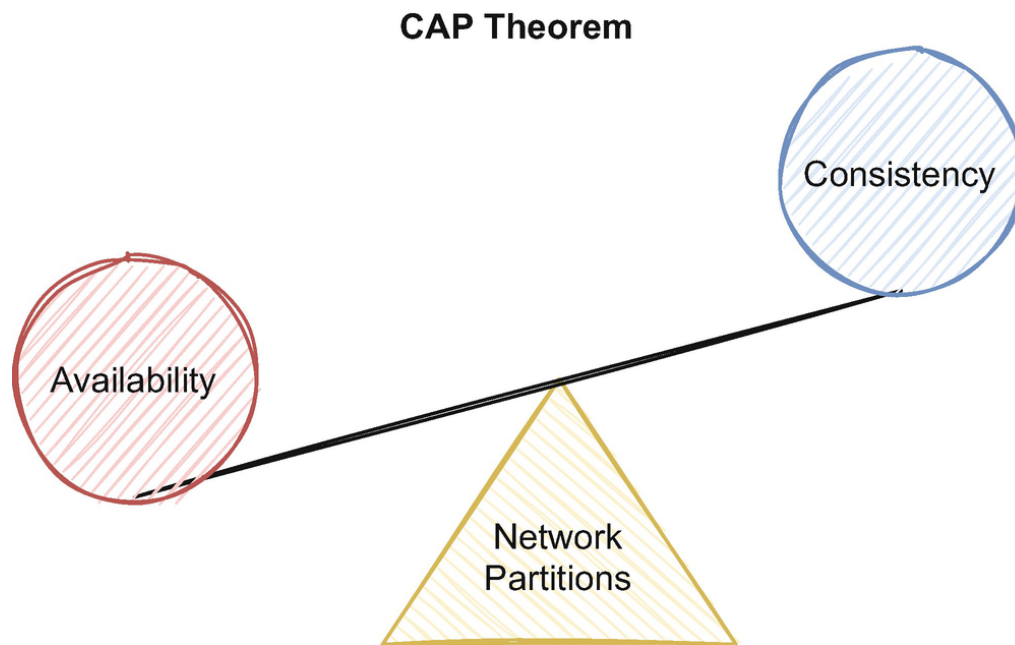


Figure 5-4 CAP theorem and the duality between consistency and availability

Distributed databases with high-availability requirements typically provide several copies of the original data in different nodes in geographically separated locations. This feature allows the system to maintain the data's integrity even when a node is lost and enjoy lower network latency for clients close to those geographical locations. However, the system has to copy the data to different nodes; the replication doesn't occur instantaneously and might produce inconsistent results across other nodes; one update that happened in one node might not yet be available for a read on a different node. Maintaining a consistent response from several nodes is even more hampered by network faults, typically known as network partitions. When a network partition occurs (due to, for example, failure in a hardware component), it can split the cluster's nodes into smaller groups that cannot communicate with each other; changes in one group's data will be invisible to the other and vice versa. The question arises on what strategy to choose, to either wait out the fault in the network and thus be unavailable or return a possible less consistent version of the data allowing the system to respond to requests.

This dilemma is illustrated by the CAP theorem, which appeared in 2000 by a keynote³ presented by Dr. Eric Brewer and greatly influenced how engineers design distributed storage systems since that time. It laid the foundations for many NoSQL databases that appeared and evolved since then and contributed to the general acceptance of weak consistency models. The reasoning behind adopting an AP (available and tolerant to network partitions) data store is very similar to the logic to adopt an asynchronous event-driven solution and lays out the fundamentals of why eventual consistency is needed. Also, the properties raised by the CAP theorem are unavoidable in any distributed system.

The CAP theorem is based on three properties: consistency, availability, and tolerance of network partitions. It states that a distributed system can either be available or consistent in the presence of network partitions. It is not possible to choose two properties out of the three since network partitions are inevitable on a distributed system; instead, you have to decide whether availability or consistency prevails. Let's dig down on those three properties:

- **Consistency:** The word consistency is broad and usually is the source of misunderstandings; in the scope of the CAP theorem, it means linearizability (differs from the consistency in ACID⁴). It guarantees that any single operation like a read or write on an entity occurs in real time, similar to what would happen if there was just one copy of the data. For example, if there are two updates on an object after one node returns the latest update, all nodes must return that update as well. It seems simple but in the context of distributed computing is a challenging problem to solve.
- **Availability :** The system is available 100% of the time, in the scope of network partitions. An important detail of this definition is that there is no limit to latency.⁵ A response is considered available even if it takes several days to return, for theoretical purposes is fitting, but for a realistic view of the real world is counter-intuitive.
- **Tolerance to network partitions:** A network partition occurs when the connectivity between two nodes gets interrupted and are unable to communicate. Network partitions can happen due to various reasons like failures in network hardware.

This definition splits distributed systems into two groups AP or CP, by either choosing availability or consistency. Due to the demanding availability requirements of Internet applications, we often see AP distributed systems as a necessary consequence of the high volume of current applications' data. AP systems typically use eventual consistency as a means

to be both available and tolerant to network partitions. Eventual consistency is also a natural consequence of the asynchronous nature of event-driven systems and nowadays is an accepted characteristic of most event-driven solutions. But should it be so lightly accepted?

The CAP theorem's availability isn't the operational availability we are used to but limited to the scope of network-related problems. Usually, we define availability as the ability of a system to respond to requests successfully. In the CAP theorem, the definition is more blurred than that. It refers to the availability of the theorem's algorithm⁶ that is closely related to faults in the network. That means 100% availability in the CAP context doesn't guarantee 100% overall availability. Several failures can jeopardize the system's availability besides network-related problems like wrong configurations, bugs on the application code, limited resources, etc. But what does that mean in real use cases? Google revealed that its Google Spanner database of all the incidents occurred only 7.6% were due to the network, and more than half were due to human-related⁷ issues like misconfigurations.

In practice, the difference in availability between CP and AP systems isn't as significant as we are led to believe; both systems can guarantee availability levels in the 99.999%; for example, Facebook's HBase⁸ CP system also reached similar levels of availability. The limited meaning of the CAP theorem's availability raises the question of whether it supports a sufficiently compelling case to adopt AP systems at the expense of consistency. This question has been raised more and more⁹ often due to the impacts of the loss of consistency in solution design.

It is undeniable that network partitions are an unavoidable concern¹⁰ and have a considerable effect in any distributed system; however, we should question if it is reason enough to blindly embrace weaker consistency models like eventual consistency.

Traditional monolith applications typically enjoyed ACID guarantees that we took for granted. Consistency was the very foundation of software and was imbued in our way of thinking and interacting with a system or database. Arguably, consistency should be handled at the database level; not doing so is pushing the responsibility and a significant increment of complexity to every client using that system. The question of whether to adopt a CP or AP system and the impact of availability in the CAP theorem's scope illustrates the same ramifications whether we

should apply an event-driven architecture to a given system. The inherent asynchronous nature of event-driven systems can and most likely will produce eventually consistent models. This nature will push the complexity to handle inconsistent results to every client using that system similarly as AP systems do. It impacts the users and other applications that interact with the system since they will have to manage the possibility of inconsistent responses leading to complex and intricate solutions to dubious user experiences or convoluted business processes. Before turning business functionality eventually consistent, we should question whether the business can effectively live with it. Either way, there are approaches we can take to minimize its impact, as we will discuss in this chapter.

5.2 Using Event Schema in Event-Driven Microservices to Leverage Eventual Consistency

In this section, you will learn how event schema can avoid eventual consistency on downstream consumers of events. Adapting event schema to your consumers' needs and real use cases is often more useful than following a strict guideline on how to design event schema. This section will illustrate with an example and discuss two different approaches to schema design.

In Section [5.1](#), we used an example of a user buying a product on an eCommerce platform. Let's drill down on the inventory and subscription service. Since most inventory management systems require audit on stock movements, they are often a typical use case to apply event sourcing. As discussed in Chapter [4](#), event sourcing can be paired with CQRS to allow more flexible reads. Figure [5-5](#) depicts inventory service with CQRS and its relation to the subscription service.

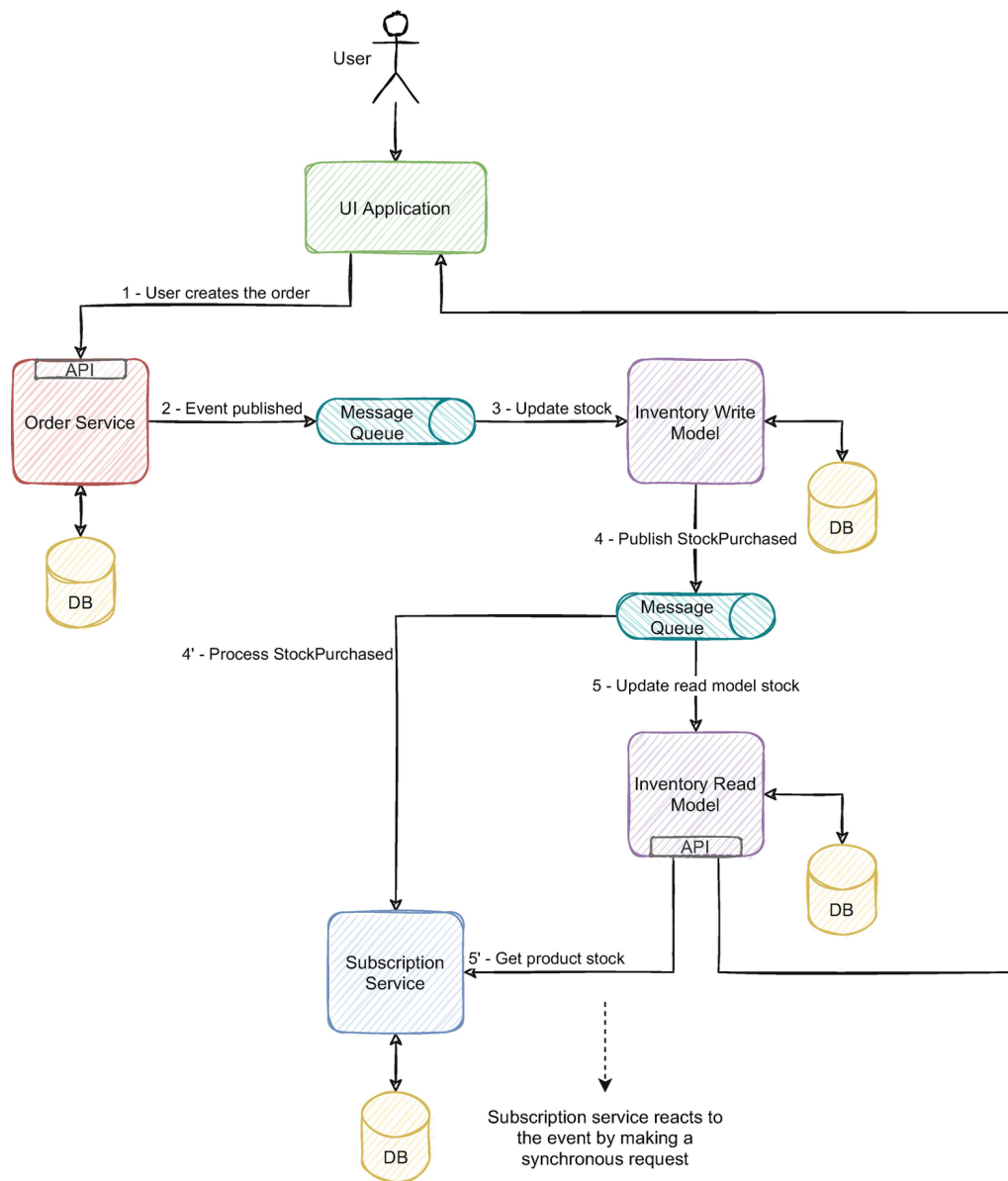


Figure 5-5 Event-driven approach mixing a synchronous request

Having the read and write model decoupled by a message queue makes the read model eventually consistent. The system will apply changes in the stock first to the write model, which is also the owner of the inventory domain logic. If the change complies with the domain rules, the service will apply the change, and an event will be published, signaling the change in stock. All interested parties can consume that event and do what their domain needs to do with that information.

A common best practice is to design events to be small and fine-grained. Fine-grained events have many advantages; performance-wise, it's faster to handle small events than large ones, serialization and deserialization are quicker and lightweight, and they have less impact on the message broker. It is also a common best practice to design events to reflect the user's intent when using a DDD approach (domain-driven design; refer to Chapter 3 for more details). This way, the events and commands

will have domain value and highlight the business flow throughout the architecture.

Applying these concepts to the example of Figure [5-3](#), we would model the stock changed event to be small and fine-grained and reflect the user's intent. Typically, stock changes occur at the product's stock-keeping unit level; for example, if the user was buying a shirt, it would be of a given size like XS. A possible event for this situation would look like Listing [5-1](#).

```
1      StockPurchased      #A
2      {
3          ProductId: 15251212,
4          Size: "XS",
5          Quantity: 1
6      }
#A Stock purchased event with minimal information
```

Listing 5-1 StockPurchased event

This event reflects the intent of the user; the user bought one product of size XS. It also is fine-grained and only has minimal information needed. Usually, these kinds of events are the easiest to publish due to all the required information being available in the publishing service at the moment of the change; the service has no additional effort to create the event.

Let's now discuss how the subscription service would consume this event. The subscription service needs to alert every user that subscribed to the alert of the product's last available stock unit. As seen earlier, the event only provides the quantity that the user bought of that size. And here lies the challenge of the subscription service domain logic; from that event, the service has no way to know the total quantity of the product. A product has several sizes, for example, they can range from XXS to XXL; the event's quantity is only referring to that specific size. The event also only provides the quantity that the user bought, not the size's current quantity.

A developer who needs to implement this functionality in the subscription service would probably fetch the current stock quantity for that product through the inventory service API. But since the inventory service read model is eventually consistent, it might return stale data. Since the inventory read model and the subscription service are consuming the same event in parallel, they will consume it at different rates. The event the subscription service is consuming might not be reflected in the read model at the time of the request. When the inventory read model returns

stale data, the subscription service will incur in one of the two faults described in Section [5.1](#).

This logic is in line with the reasoning of NoSQL databases adopting eventual consistency and delegating the complexity of achieving strong consistency to the applications that use them explained in Subsection [5.1](#). In this case, the choice of fine-grained schema on inventory service delegates the complexity of handling stale information to the consumers. If we deal with this only on subscription service, it will add significant complexity to the service to solve a purely technical and not business-related issue.

Most resources advise to create small fine-coursed events, mostly due to performance and other considerations. However, in the real world, what most teams find out is as the microservice ecosystem grows, so this concern becomes more pressing. One highly beneficial approach is to approach it by designing event schema with consumers' needs in mind.

Going back to the inventory and subscription service example as seen in Figure [5-3](#), since inventory service has the stock information for all sizes, it would be arguably easy to send an event with additional details, illustrated in Listing [5-2](#).

```
1      StockPurchased  #A
2      {
3          ProductId: 15251212,
4          Size: "XS",
5          Quantity: 1,
6          ProductCurrentQuantity: 5,
7          ProductSizes: [
8              {
9                  Size: "S",
10                 Quantity: 3
11             },
12             {
13                 Size: "M",
14                 Quantity: 2
15             },
16         ]
17     }
#A Stock purchased event with information adapted to the
consumer needs
```

Listing 5-2 StockPurchased event denormalized

The property indicating the product's current stock information for every size would greatly ease the implementation in subscription service because it would be able to know the product's total stock instead of a sin-

gle size. Instead of requesting the current stock information from an eventually consistent model, the service would process its domain logic by relying solely on the event's data. Besides removing a great deal of complexity from the subscription service, this approach also gives it a performance boost since it avoids the need for a network request.

Adding additional information to the event is a good strategy when the information is available but depends on the inventory service design. The aggregate size becomes a pivotal decision in this situation:

- If we design the aggregate in the inventory service to be the product, it would be trivial to obtain the total product's stock.
- If we design the aggregate in the inventory service to be the product's size, it would require a more complex solution. Since the product stock comprises each size's stock information, we would require interaction between several aggregates. To obtain this information, we would need to apply a more complicated solution like a process manager. We further detail this topic in [Chapter 8](#).

Adapting the event schema to consumer applications' needs can be one of the most powerful patterns you can use to avoid eventual consistency. It easily contains the complexity of the architecture and the applications while retaining the advantages of event-driven systems.

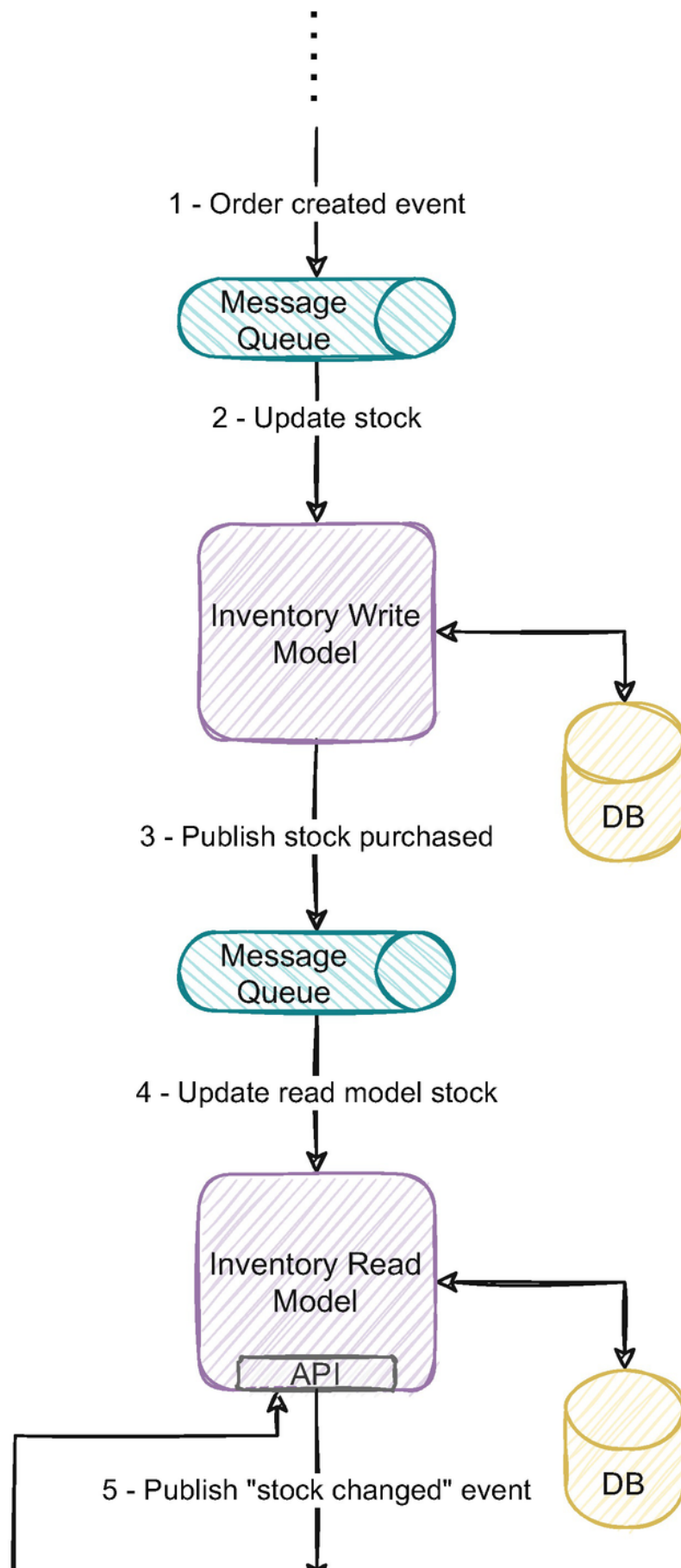
The full implications and additional alternatives to event schema are also discussed further in [Chapter 8](#).

5.3 Applying Microservice Domain Boundaries to Leverage Eventual Consistency

In this section, you will learn to contain the impact of eventual consistency by segregating the services into domain boundaries. Bounded contexts are a core concept in DDD and you can use the same rationale to an event-driven architecture, especially when the system starts to have a high number of components. Using a boundary organization enhances the autonomy of the teams working in those components and can contain the impact of eventually consistent models. This section will detail how domain segregation can control that impact, illustrating it with an example.

In the last section, you learned how to solve the challenge of fetching stale data from an eventually consistent read model by adapting the event schema. We can use

additional options to solve the need to fetch additional data; using the same example we used before with a user buying a product on an eCommerce platform, we can simply have the subscription service consume an event after the read model is updated. By changing the sequence of consumers between the services, we guarantee that the model is up to date with the event's information, illustrated in Figure 5-6.



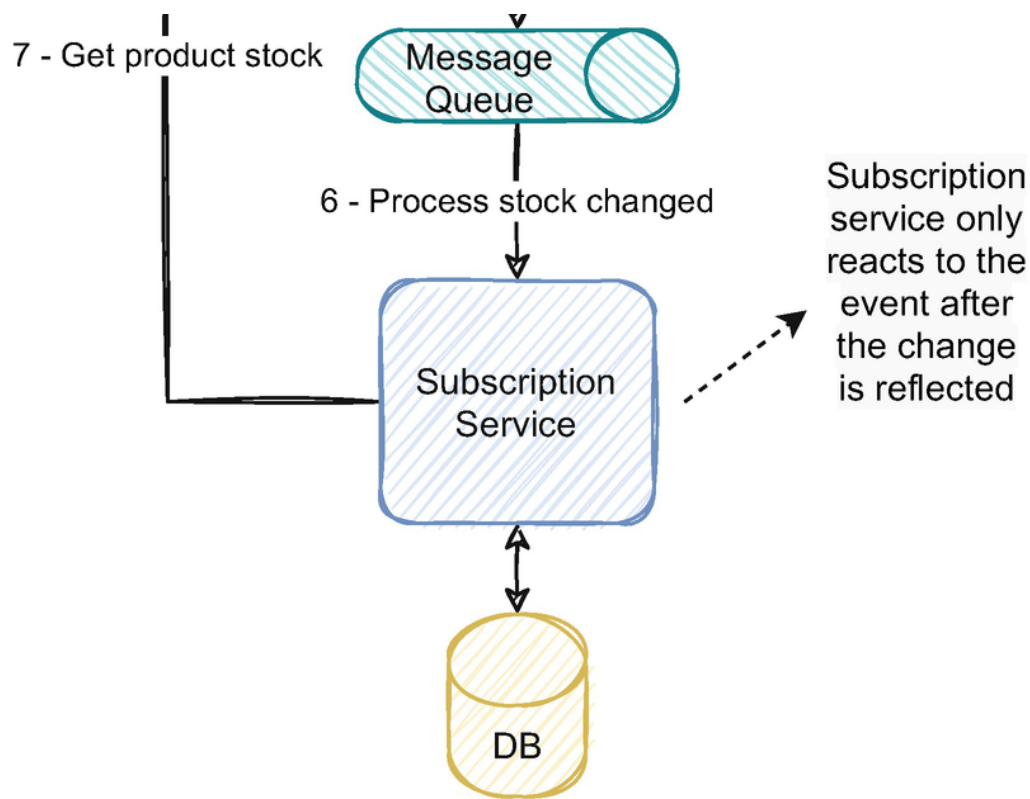


Figure 5-6 Subscription service processing the change only after it is reflected on inventory read model

This approach also solves possible inconsistency issues by informing the user too early if the subscription service would notify the user with a link to the product. If the user accesses it while inside the inconsistency window, the user would see incorrect stock. Changing the sequence of components, the subscription service would only notify the user when the data is updated.

However, this solution has some pitfalls:

- It is fairly debatable whether a read model should send an event. If we look at it in purely DDD terms, only a change in an aggregate should trigger an event; in this case, the only event that should be published would be from the write model that is the owner of the domain. It is arguably justifiable if it is not an event just a notification that the read model is up to date. However, it usually is a sign that something is out of place (in this case, we should avoid the synchronous call).
- You should use this approach sparingly and inside a specific context. Using it often will rapidly increase the complexity of the overall architecture and the flow of the information. The architecture should have clear boundaries and manage these issues inside each boundary. The data should flow between the boundaries and not have a tangled flow between the read models and the domain owners.

- The overall time to reflect the change throughout the system will increase.

In DDD, a bounded context is the contextual relationship of a specific domain model. It usually represents a subset of the domain of the overall system with its individual domain model. It's also segregated from other bounded contexts with clear boundaries between them. Although the bounded context refers exclusively to the domain model, in a microservice event-driven architecture, one bounded context can span various individual components. For example, when applying CQRS, we can have one component responsible for the write model and another component responsible for the read model. They both use and represent the same domain, so the two components are part of the same bounded context.

In a growing microservice architecture, it's important to understand the system's bounded contexts and which services belong to each one of them. Having clear domain boundaries between slices of the system will facilitate the evolution of the business. The decoupling between boundaries is the stepping stone to the seamless business evolution and limiting the technical impacts of that evolution. We detailed DDD and domain boundaries on an event-driven architecture in [Chapter 3](#).

An alternative solution for this problem could be defining two separate boundaries, one for the inventory domain and another for the subscription domain. This separation has high synergy with a DDD approach, forcing you to understand the business and think about what kind of boundaries make sense and where to draw the boundaries. The business process is the flow between all needed domain boundaries. This is similar to the event choreography pattern detailed in [Chapter 4](#). The anti-corruption layers guarantee that concepts from a given domain don't leak to another domain. [Chapter 3](#) details this concept. We illustrate this solution in [Figure 5-7](#).

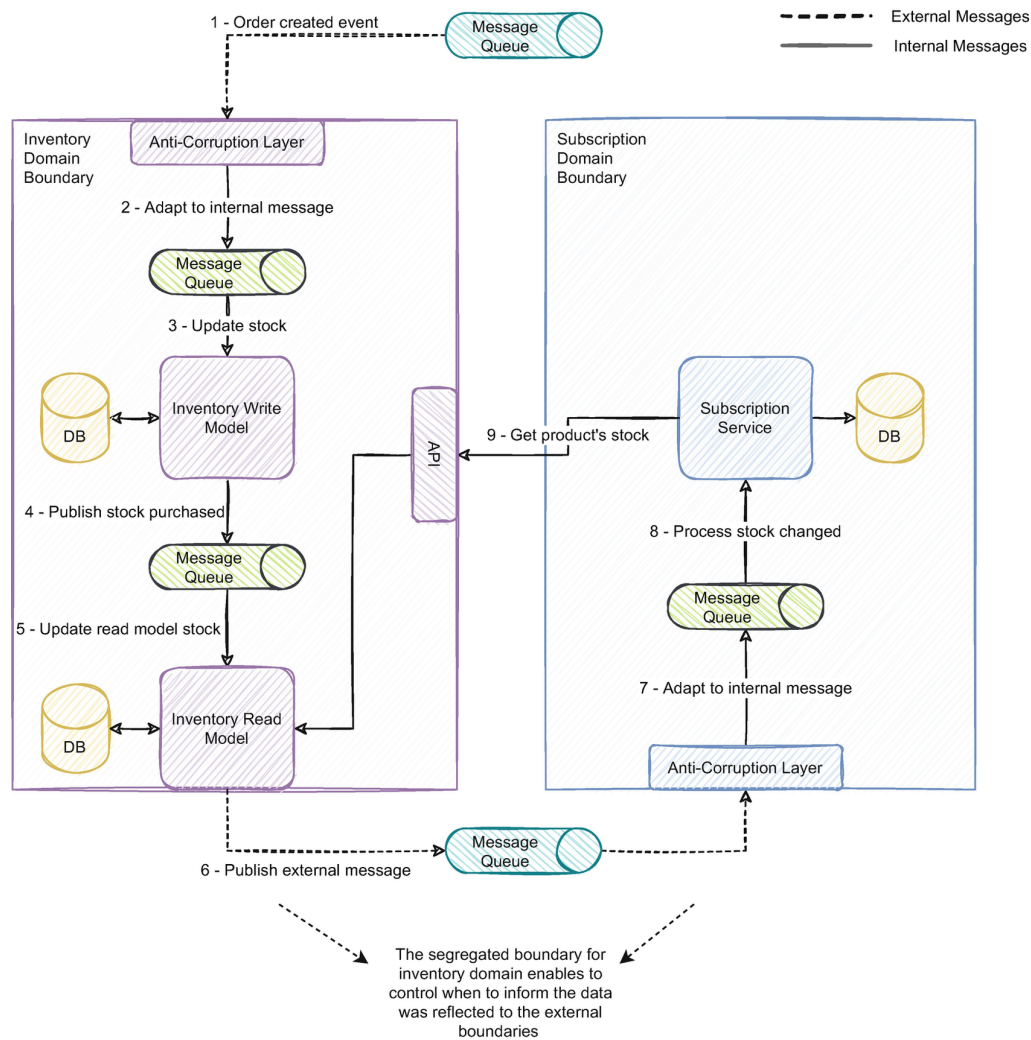


Figure 5-7 Applying domain boundaries to the same use case

This solution is a more structured way to evolve the domains without losing track of the overall flow. The order created event is forwarded inside the boundary by the anti-corruption layer that adapts the event to keep only the needed information for that boundary. Inside the boundary, the inventory services will process the event accordingly with their logic. The write model will change the aggregate's stock and publish an internal event. The read model will use the internal event to update its information and publish an external event only when the data is reflected in the read model. The complexity inside the inventory boundary is abstracted from external boundaries, like the subscription boundary, and can evolve as needed without impacting other boundaries.

Each domain can change; we can add or remove more components, but the business process's overall flow will remain the same. We also deal with eventual consistency inside the boundary; changes that happen are notified to external consumers only when all the services inside the boundary processed it. This feature contains eventual consistency for ex-

ternal consumers; this way, we can guarantee the change the consumer is reacting to is already propagated inside the domain.

5.4 Handling Eventual Consistency Delays with Event Versioning

In this section, you will learn how to use event versioning to understand delayed responses and possible actions to mitigate its impacts. The most direct approach to eventual consistency is simply to try the operation later when it is no longer eventual. This solution is very straightforward and would probably be the most obvious alternative to deal with temporarily stale data. However, it has two not so obvious caveats: knowing the data is not up to date and when you should use it. We will illustrate this solution with an example and discuss possible approaches to avoid these caveats.

There is a fascinating article¹¹ named “The Tail at Scale” from Jeffrey Dean and Luiz André Barroso detailing how Google managed to obtain 99.9% latency SLA on some of their services by optimizing the slowest requests on the 95th percentile. Distributed systems typically have latency fluctuations beyond the scope of the service. In a perfect world, we would avoid those fluctuations entirely, but these fluctuations are unavoidable in a complex distributed system, much like the fault-tolerant techniques we often employ. The article details several tail-tolerant methods to minimize or mask the impact of these temporary latency spikes. Some of them rely on retrying requests to the replicas when the original request takes longer than a specified time. The retrying strategy has a higher resource consumption due to the additional requests, which, the article explains, can be as low as just 2% with the right approach.

We can draw an analogy from this article to retrying strategies in an event-driven architecture. Event-driven services should keep up with the message load and not lag behind the changes that occur in the system (as discussed in Section 5.7). However, the services will likely lag eventually one time or another due to unforeseen peaks in load, but that should be the exception rather than the rule; it should be the curve’s tail. If this is in fact the case, we can argue that we can opt for an optimistic approach; we can assume there isn’t eventual consistency. When a request fetches stale data, it can react to it with a retry.

Using the same example with the inventory and subscription service, when requesting the information to the inventory service if the data is stale, the service should make an additional request after a predefined amount of time. However, this assumes that the inventory service is faster and is usually up to date; the retry would be the exception.

The first caveat is how to know the data is stale. To solve that, we can use event versioning; each event should have a version (it is also essential to manage idempotency detailed in Chapter 7) that identifies the entity's version. In DDD terms, it could be the aggregate version; for every change that happens on the aggregate, the service increments the version. It can also be a timestamp, which is more practical, although not sequential. This version should also be available on the API; the subscription service, when making the request, would compare both versions; if the version of the event were higher than the version of the API, the service would need to retry the request. This means the service didn't reflect the change in the event on the read model yet.

Retrying the requests is the most straightforward strategy you can do but is also riskier. We often hear how retrying strategies can improve resilience, however, more often than not it sets the system up for failure. We can apply it only in less critical components or as a quick bandage. The consequence is that the service is considerably more susceptible to its dependencies' load peaks. The implementation should have a circuit breaker and a backoff strategy. Used throughout, the architecture will make the system brittle and vulnerable to cascading failures. Retrying is the symptom of the lack of a more sustainable approach; we should use it wisely and sparingly.

There is also the issue of what to do when all retries fail. I saw some services that simply failed the message processing or delegated the failure to a business process, like a manual operation. Although they might be acceptable in some contexts (e.g., some less critical use cases like logging), I find these kinds of approaches disheartening and half-hearted. There are some compensation alternatives like scheduling the handling of the failure to a later date; ultimately, consistency is well, eventual, so at some point in time, the lag will clear.

Ordering is also an important consideration; retries make sense when message consumption is associative, commutative, and idempotent. Oth-

erwise, the service might not be able to retry with consistent results. For more details, refer to Chapter [6](#).

Event versioning isn't a definite solution on its own, just a tool that we can use to manage delays. We should apply it as a tactical solution for an urgent matter; more often than not, we require a more sustainable and reliable solution like the others mentioned in this chapter.

5.5 Saving State in Event-Driven Microservices to Avoid Eventual Consistency

In this section, you will learn how to prevent eventual consistency by storing state external to the service. Typically, microservices are faced with eventual consistency due to the hybrid nature of most real-world architectures. Some services react to events, and others make synchronous calls depending on the use case. By storing state, we can spare the synchronous call, avoiding dealing with an eventually consistent model.

Using the same example with the inventory and subscription service, depicted in Figure [5-5](#), the issue with eventual consistency rises due to the synchronous call between the subscription service and the inventory read model. To avoid doing the synchronous call, we could store the product's stock internally in the subscription service. When the subscription service needs the stock information for all product sizes, it could fetch it locally, as in Figure [5-8](#).

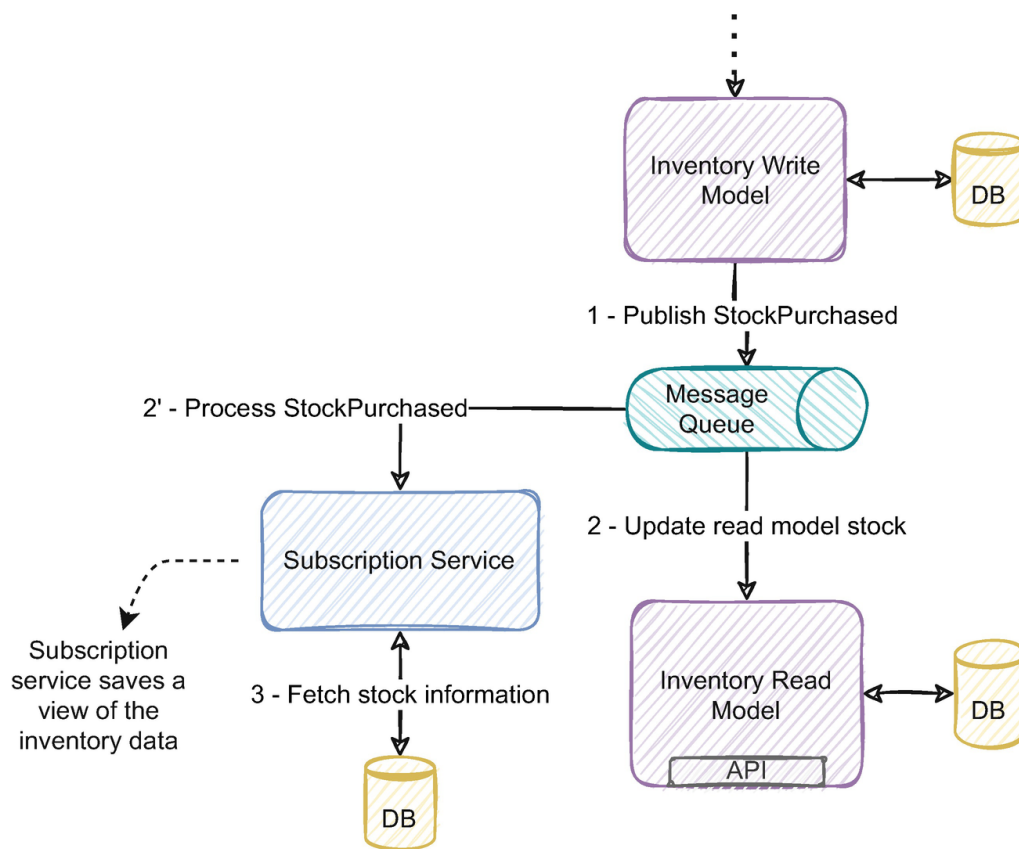


Figure 5-8 Storing external state to avoid the synchronous request

This approach is the most adherent to the event-driven mindset. By removing the synchronous request, the service becomes entirely decoupled from the dependency. It becomes more resilient since it won't be affected if the inventory read model is unavailable. Performance-wise is also better since fetching the database's data is often faster than a remote network call. Similarly to the solution adapting event schema, the subscription service is only dependent on the upstream inventory events.

However, the subscription service must maintain and internally store all the inventory information. When there is a large amount of data, storing it will impact the infrastructure and its costs. The subscription service will also need to maintain that data up to date, which has a considerable development overhead than just requesting the inventory read model data.

This approach gives precedence to the service's autonomy and performance. It is a useful alternative to solve this kind of challenge; however, we must avoid the pitfall of applying it blindly in every use case. Otherwise, several copies of the service's data will spread throughout the architecture, becoming hard to manage and maintain. We discuss further the tradeoffs of using events to provide state transfer in [Chapter 8](#).

5.5.1 Buffering State As an Alternative to Persistence

We discussed how storing state could avoid the dependency of synchronous requests to an eventually consistent model. We also discussed the drawbacks of storing that data. An alternative to persisting the state is buffering it to memory. Storing every external state a service needs as an internal view of that state can be difficult to manage, to initialize, and to maintain up to date. An alternative to persisting the state is to load it in memory. There are two strategies to achieve this:

- **A predefined buffering window:** The service keeps in memory all events that arrive inside that window's timespan. This strategy is relevant when a use case applies a batch of operations to the same or related entities in a small time frame, for example, bulk actions or uploads of information. If the service needs the data from the entity it is processing or a related entity, it can load directly from that buffer. This strategy's success is directly related to the timespan's size and the likelihood of the entity changing in that timespan. There's always the chance the service doesn't find the needed entity in memory and might have to fall back to a request, thus being limited to specific use cases. This strategy is also useful in data-intensive applications that write to data stores. Usually, a bulk change is faster than several single changes; the buffer can help cluster changes and update them on a single operation.
- **Loading the full state:** The service can load all the dependent states to memory. This solution implies all the event streams are available on the event broker. The service can read all the information from the beginning of the topic and keep it in memory. Other events would keep the in-memory view up to date. This strategy is only relevant to small data sets that aren't expected to grow in the future. For example, if the subscription service would alert users only for products in the "shoes" category, it might need the product's category information. Assuming there won't be more than a few hundred categories in the platform, the service could load them to memory and use that view when needed. The service needs to load all the information on startup, which can be too cumbersome in large data sets.

Figure [5-9](#) illustrates this solution applied to the same example using the subscription and inventory service. The subscription service loads the events in memory and, instead of making the synchronous request to the inventory read model, uses that information. This way, the service can decide whether it is the last stock unit left and if it should alert the user.

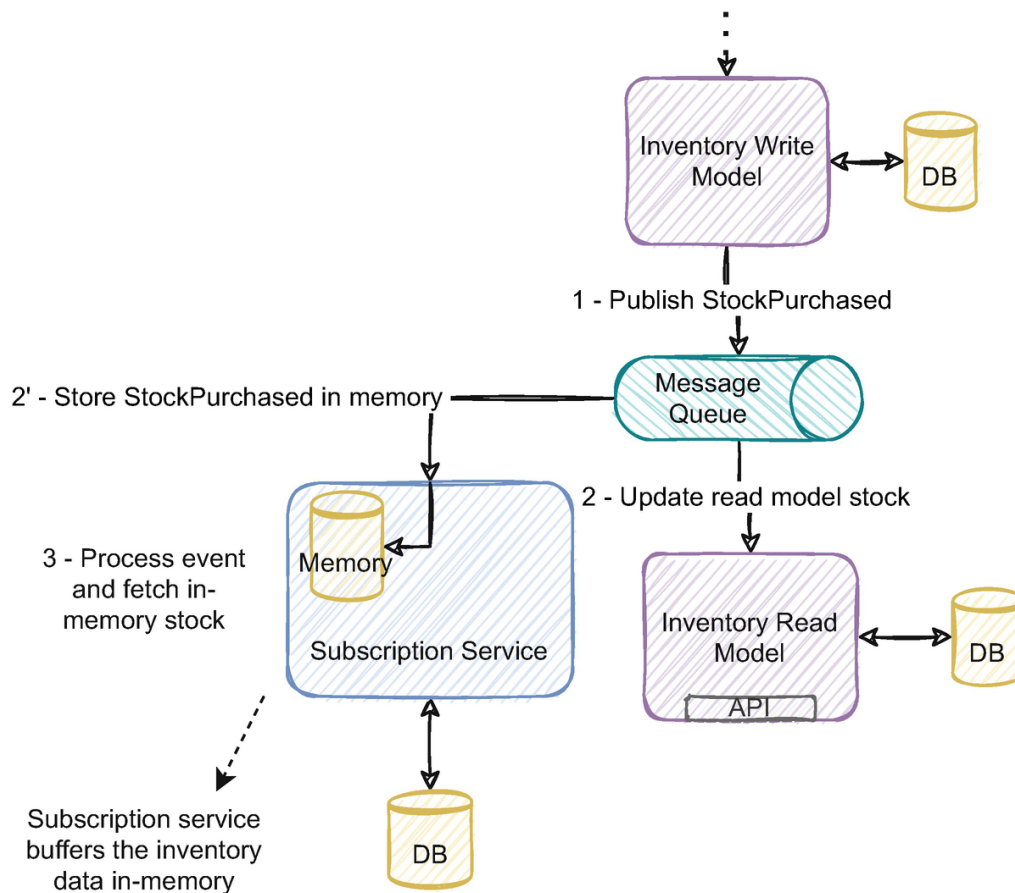


Figure 5-9 Subscription service saves the stock event stream in memory

However, this alternative depends on whether the subscription service has all the information of all sizes in memory. In this case, a buffering window would only be an adequate choice if the system changes all sizes simultaneously; otherwise, most likely the service won't have all the information needed. If we read the event stream and load all stock information to memory, we would address that problem. However, it would only make sense if the stock information isn't excessive; otherwise, we risk occupying a significant percentage of the service's memory.

By keeping the information in memory, the service doesn't need to request the information or persist it in a data store, making it easier to manage and contain costs. However, this is only relevant for small data sets; use cases with or more than hundreds of thousands of entities are better off by either persisting the information in a database or using another alternative. This alternative is also the most performant since all information is in memory; it is faster than fetching it from the database or doing a remote request.

5.6 Tackling Eventual Consistency with the End-to-End Argument: A Real-World Use Case

In this section, you will learn how to reduce the impact of smaller, eventually consistent parts of the system by guaranteeing the end-to-end flow's correctness. You will learn what the end-to-end argument is and how we can use it in an event-driven architecture. We will illustrate its application by using an example of an eCommerce platform.

Until now we discussed a retail eCommerce platform that sells products to a general audience. When buying a product on that platform, the user creates an order which is processed by the order management boundary. To complete the full order workflow, several other boundaries would have to handle the event and process accordingly to their own domain; for example, the order boundary would publish an event signaling an order was created that would be consumed by the pricing boundary to apply the relevant taxes on the order, then the inventory would update the stock information, and so on. The product boundary would aggregate all the product information and provide read models to search on several product properties. By accessing the UI application, the user would be able to request information from these read models. Figure [5-10](#) illustrates this example.

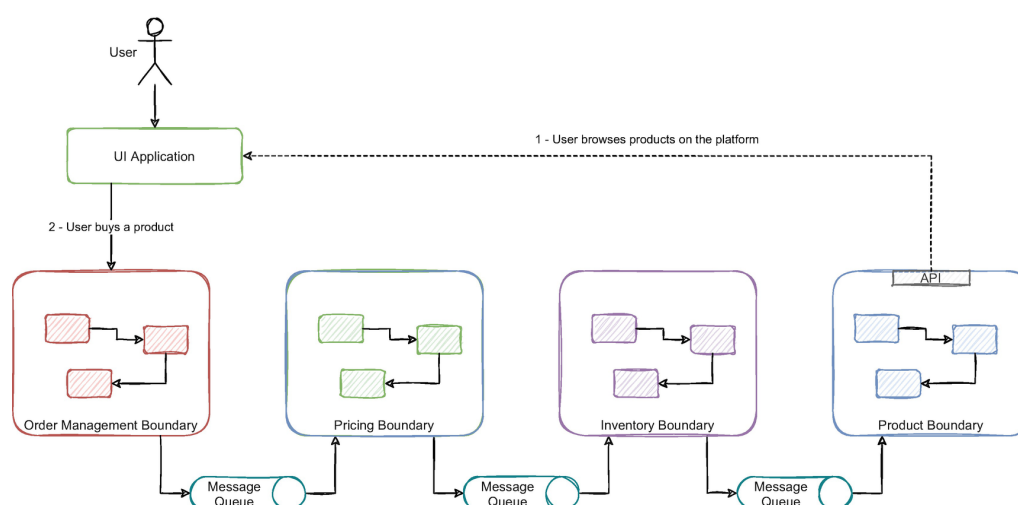


Figure 5-10 User browses and buys a product on an eCommerce platform with several event-driven domain boundaries

By reaping all the benefits event-driven provides, the product read models are eventually consistent. When the user browses the platform's product catalog, it might see inaccurate information due to the data's eventually consistent nature. The users might even try to buy products that don't have stock anymore because the upstream services are still processing them. We can take several approaches; an interesting approach in this situation is the end-to-end argument.

The end-to-end argument was first articulated in an exciting paper^{[12](#)} by Saltzer, Reed, and Clark in the context of computer networking design. In it, they described how strong guarantees at low levels of the system

might be hard to implement and have a negligible impact by providing them at that low level. The reliability guarantees could be implemented in the end-to-end flow rather than in smaller, low-level components. Doing so opens the opportunity for those smaller components to have more straightforward and more performant approaches by forfeiting those stronger guarantees.

In the paper, the authors describe the use case of a file transfer. There are several intermediate components between the transmitting and receiving computer to transfer one file between two computers. Each of those components can be the victim of several types of faults like system crashes or network faults. One can argue that to achieve a careful file transfer, we need to guarantee each component's resilience and endow each component with retry strategies and duplicate copies to ensure it is fault-proof. However, providing those kinds of guarantees to each component will significantly harm those components' performance and complexity. Besides, guaranteeing the fault tolerance in one component doesn't ensure the end-to-end file transfer success. Thus, the implementation overhead of being exceptionally reliable in a single component does not reduce the end-to-end functionality burden to guarantee reliability.

To solve the issue for the use case in Figure [5-10](#), we could try to force the product boundary to be strongly consistent and provide consistent results. Every time any change occurred in the system, it would have to be reflected instantaneously on the product boundary. That strategy would also mean complex interactions and forfeiting decoupling between every other boundary and the product boundary. It would likely mean every other component of the workflow would have to guarantee consistency across boundaries, so each one provides consistent results all around. In a distributed system, this situation would mean sacrificing performance, availability, and the ability to scale seamlessly. It would also significantly increase the complexity of each component.

The end-to-end argument draws a compelling proposition that is applicable in a distributed microservice event-driven architecture. Instead of increasing each component's complexity substantially, we could loosen the constraints on intermediate components of the flow as long as we guarantee the end-to-end flow's consistency. In this concrete example, illustrated in Figure [5-11](#), we could maintain the decoupling between boundaries and keep the product boundary eventual consistent as long as at the moment of the purchase, the system would have a way to guarantee the information's consistency, for example, the product having stock.

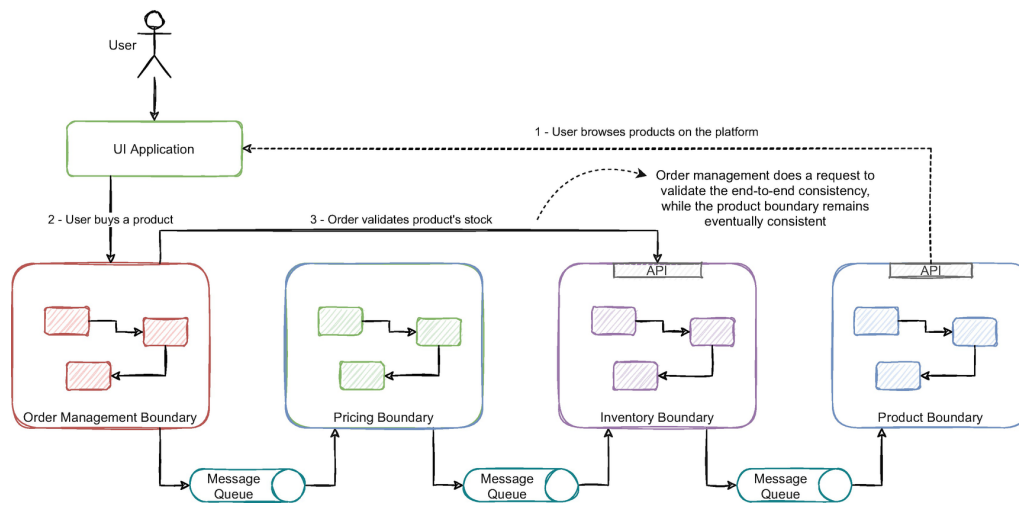


Figure 5-11 End-to-end validation of the product's stock

The order management system would validate the conditions of the order before advancing with the flow. This example is the typical situation where you are browsing items on a website and, although the UI displayed that item as having stock, the checkout might fail because the UI wasn't up to date anymore. This way, the component responsible for aggregating information from several different sources the UI needs (product information, stock, price, descriptions, etc.) can enjoy the decoupling and performance boost for not guaranteeing strong consistency, as long as there is an end-to-end process that ensures the business rule is satisfied.

5.7 For Most Use Cases, It's Not Eventual If Nobody Notices

In this section, we will put eventual consistency into perspective and understand how the business can coexist with it without considerable impact. We will discuss eventual consistency in NoSQL databases and detail how we can apply the same reasoning to event-driven systems.

As we discussed in Subsection 5.1, eventual consistency provides no safety guarantees. The very meaning of eventual is daunting; it states that sometime in the future the system will return a consistent result, but it doesn't say when. It is hard for a business to come to terms with such a vague notion of something so pivotal as consistency. It is also hard to manage the users' expectations; an ordinary user expects changes to be instantaneous. It doesn't expect them to take an indefinite amount of time to be reflected throughout the system, and surely explaining the CAP theorem to them isn't the solution. Considering this, it might sound odd how

NoSQL databases adopted eventual consistency and why it is widely deployed to production.

In NoSQL databases, it is observed that in practice, with real-world use cases, eventual consistency looks like strong consistency. Some studies show that the inconsistency window in eventually consistent databases is small and can often be neglected. For example, one study¹³ found that about 98% of requests have an inconsistency window between zero and one millisecond when within a single availability zone for Cassandra. For MongoDB, the same study shows an inconsistency window of 5ms or less in 96% of requests. Another study¹⁴ showed that out of the curve requests, Cassandra's inconsistency window was almost always less than 200ms.

As we discussed in Subsection 5.1, it is fairly debatable if we should settle for eventual consistency in databases. But the fact they are widely adopted, and we can see eventual consistency as good enough for many use cases, raises the question of why. It is because, in practice, the inconsistency windows in these use cases are small. So small most often are deemed insignificant.

Eventual consistency is a consequence of event-driven systems. The best way to deal with it is by designing the system to be fast and guaranteeing the inconsistency windows are small. As proven by eventually consistent databases, if eventual consistency is fast enough, it appears strongly consistent, having a neglectable impact. Eventual consistency doesn't need to be slow; if it is fast enough to appear strongly consistent, as far as users and developers are concerned, it isn't eventually consistent.

What does it mean to be fast on an event-driven microservice? We need to build two fundamental characteristics from the ground up: the ability to scale and meaningful metrics. As discussed in Chapter 1, we design microservices to be autonomously developed and individually deployed. Also, they should be horizontally scalable. By providing these characteristics on an event-driven microservice, we can scale it on demand to face unforeseen peaks in load. We need meaningful metrics to react to the system's delay and use them to deploy new instances. One important metric is the number of messages waiting to be processed; if it is large, likely the inconsistency window also is. The best way to guarantee this happens seamlessly is to have an autoscale system in place and use it to react to unforeseen load peaks. When a metric, for example, the con-

sumer lag, is above a given threshold, new instances will be deployed. We will work through a practical example in Subsection [5.7.1](#).

If we guarantee the size of inconsistency windows isn't a concern; we can reduce the impact of eventual consistency with the best approach possible, by living with it.

5.7.1 Event-Driven Autoscaling Use Case with Prometheus and Kafka

In the last section, we discussed the importance of having a small inconsistency window to reduce the impact of eventual consistency. We also discussed how an autoscale system could help achieve this. In this section, we will explain how we could implement an autoscale system using Prometheus, Kubernetes, and Kafka.

In the example in Figure [5-5](#), the subscription service uses stock events to understand if the product has only one stock unit left in order to alert users. If the team managing the product's stock does a massive stock import, it is very likely that the subscription service lags behind the unexpected load of messages. If we have a system in place to monitor the subscription service consumer lag (e.g., Prometheus and Kafka Exporter), we could react to the delay by deploying new service instances. We can make the detection and deployment of new instances automatic (e.g., Kubernetes and HPA). In Figure [5-12](#), we illustrate this example.

Relevant technologies mentioned in Figure [5-12](#):

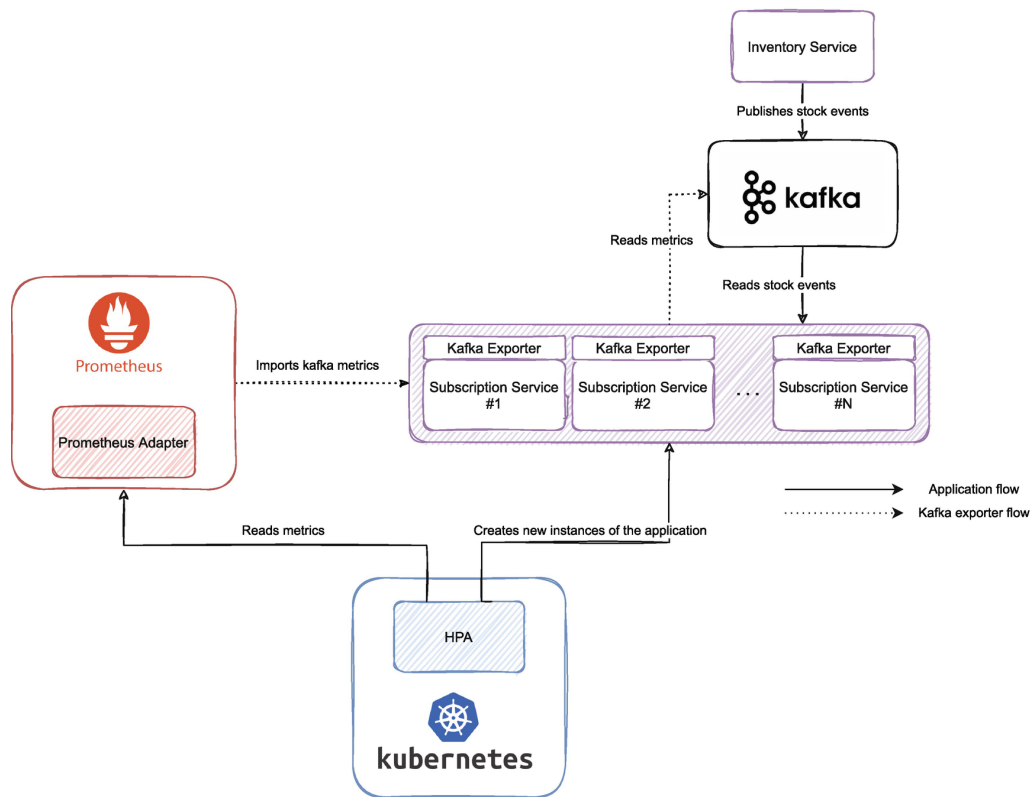


Figure 5-12 Autoscaling subscription service example with Kubernetes, Prometheus, and Kafka

- **Kubernetes** : A platform for managing containerized services. This example will use it to read the relevant metrics for scaling and horizontally scale the application with new instances.
 - **HPA (horizontal pod autoscaler)** : Used by Kubernetes; automatically scales the number of instances of the application.
- **Prometheus** : Systems monitoring and alerting toolkit. This example will use it to import Kafka's metrics and make them available to HPA.
- **Kafka** : An event streaming platform; can be used to publish and subscribe to streams of events. We will use it as the event broker where the service reads and publishes messages.
 - **Kafka Exporter**: Application to extract metrics from Kafka (e.g., topics and consumer lag) and expose them.

In the example, Kafka Exporter reads metrics from Kafka and exposes them to Prometheus. The most relevant metric in this example is the consumer lag; we want to deploy additional instances of the subscription service when the lag goes above a given threshold. Prometheus creates custom metrics for Kubernetes; we used Prometheus in this case because it is a powerful monitoring tool that can be combined with several other metric capabilities and exposes simple functionality to configure custom metrics. HPA will use the custom metrics exposed by Prometheus to add additional instances of subscription service when needed.

Subscription service lag will increase when inventory service publishes an unforeseen load of messages. When the lag reaches the specified threshold, the HPA will deploy additional subscription service instances to consume the lag faster. When below the threshold, it will remove the additional instances. Since there isn't a throughput higher than expected and we dealt with the load peak, there is no need to oversize the service. So the HPA will automatically deallocate the resources, guaranteeing the additional cost for the other resources occurs only on the peak. This use case is one example of how we can use autoscaling to maintain the system fast and guarantee that the services don't lag behind the changes the users do on the system.

5.8 Discussing the Tradeoffs of Typical Eventual Consistency Handling Strategies

In this section, we will discuss the tradeoffs of each solution we discussed until now and quickly sum up their advantages and limitations. This quick sum up will make it easier for you to weigh the tradeoffs when applying to a given use case.

In this chapter, we discussed several patterns of dealing with eventual consistency in several different use cases. Each of them has advantages and limitations, summed up in Table 5-1. The use cases you will find in real life sometimes will be easy to understand which one is the most adequate for that situation; some use cases won't be so easy. Most of the time, several solutions can be applied to a use case making it hard to know which one to apply. Most of the time, there isn't a perfect fit; it is about weighing the tradeoffs and making the best decision with the information we have.

Table 5-1 Tradeoffs of the different patterns

Pattern	Advantages	Limitations
---------	------------	-------------

Pattern	Advantages	Limitations
Denormalized event schema	Straightforward to implement Higher decoupling Better performance due to the removal of the external request Scaling is more linear due to the removal of a dependency	Larger events have a performance hit on the broker and producing and consuming messages (although much lesser than the external request) The information has to be available on the upstream service; otherwise, it will significantly increase the complexity Events might lose their meaning if they become too generic
Saving state	Future use cases might benefit from the local data Higher decoupling Better performance due to the external request removal Scaling is more linear due to the dependency removal More resiliency due to the dependency removal	Increase in storage cost Development and maintenance overhead Applied throughout the architecture, data will be copied several times; substantial changes to the source system schema might impact several services

Pattern	Advantages	Limitations
Applying domain boundaries	None or little developments needed, just a clear strategy Often comes for free if boundaries was a concern since the beginning	Overall time to process the flow will increase Without boundaries and a clear strategy to manage the flow, it will soon become complex and hard to track
Event versioning	Straightforward to implement Useful when in need of an easy fast solution to an incident or urgent requirement	Doesn't solve the eventual consistency; it's just a band-aid Without compensation actions might fail with load peaks With compensation actions substantially increases the application complexity Impacts performance
End-to-end argument	Easy to implement Gives the tools for intermediate components of the flow to be flexible about their consistency guarantees Better performance on the overall flow	It is often needed a synchronous way to validate the business rule

5.9 Summary

- Eventual consistency can impact the business, and we need to weigh the tradeoffs between using an event-driven or a synchronous approach.

- The tradeoffs between availability and consistency illustrated by the CAP theorem are significant when designing event-driven solutions, although the CAP theorem itself has limits.
- We can adapt the schema of events to avoid synchronous calls to eventually consistent read models, thus avoiding processing stale data.
- We can organize and structure our services so that downstream services don't need to deal with eventual consistency, having in mind the flow of the business process.
- Event version and entity versioning are essential to understand if the service is processing stale data. Retries can be an alternative to deal with delays, although we often need a more sustainable approach.
- Saving a view of the external information can avoid the need to request synchronous information from eventual consistent read models. We can further optimize this by keeping the information in memory for small data sets.
- With the end-to-end argument, instead of increasing each component's complexity substantially, we could loosen the constraints on intermediate components of the flow as long as we guarantee the end-to-end flow's consistency.
- Eventual consistency doesn't need to be slow; if it is fast enough to appear strongly consistent, as far as users and developers are concerned, it isn't eventually consistent.

Footnotes

1 Check this article by Jepsen which details the different degrees, "Consistency Models," <https://jepsen.io/consistency>

2 These two properties were first described in this article by Bowen Alpern and Fred B. Schneider, "Defining Liveness," October 7, 1985

3 See Dr. Eric A. Brewer, "Towards Robust Distributed Systems," July 19, 2000, <https://people.eecs.berkeley.edu/~brewer/cs262b-2004/PODC-keynote.pdf>

4 Further details in "CAP Twelve Years Later: How the "Rules" Have Changed," May 30, 2012, www.infoq.com/articles/cap-twelve-years-later-how-the-rules-have-changed/

5 See Martin Kleppmann, "A Critique of the CAP Theorem," September 18, 2015, <https://arxiv.org/pdf/1509.05393.pdf>

- [6](#) See Martin Kleppmann, “A Critique of the CAP Theorem,” September 18, 2015, <https://arxiv.org/pdf/1509.05393.pdf>
- [7](#) Full article in Eric Brewer, “Spanner, TrueTime & The CAP Theorem,” February 14, 2017, <https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/45855>
- [8](#) Full article in Rishit Shroff and Zelaine Fong, “HydraBase – The evolution of HBase@Facebook,” June 5, 2014, <https://engineering.fb.com/2014/06/05/core-data/hydrabase-the-evolution-of-hbase-facebook/>
- [9](#) See Robert Yokota, “Don’t settle for eventual consistency,” February 17, 2017, <https://yokota.blog/2017/02/17/dont-settle-for-eventual-consistency/#fn-517-10>, and Ben Darnell, “The Limits of the CAP Theorem,” June 27, 2017, www.cockroachlabs.com/blog/limits-of-the-cap-theorem/
- [10](#) Further details in “The network is reliable,” June 2, 2013, <https://aphyr.com/posts/288-the-network-is-reliable&ved=2ahUKEwjY7sjEvKfsAhVOcBQKHf78AxIQFjAEegQIAxAB&usg=AOvVaw3159NbJwFhJotfpzh8iCBR&cshid=1602245361376>
- [11](#) Full article in Jeffrey Dean and Luiz André Barroso, “The Tail at Scale,” February 2013, <https://cacm.acm.org/magazines/2013/2/160173-the-tail-at-scale/fulltext>
- [12](#) Full paper in J.H. Saltzer, D.P. Reed, and D.D. Clark, “End-to-end arguments in system design,” May/June 1998, <https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf>
- [13](#) Full paper in David Bermbach, Liang Zhao, and Sherif Sakr, “Towards Comprehensive Measurement of Consistency Guarantees for Cloud-Hosted Data Storage Services,” August 2013, www.researchgate.net/figure/Distribution-of-Inconsistency-Windows-in-MongoDB_fig5_259540354
- [14](#) Full paper in “Toward a Principled Framework for Benchmarking Consistency,” October 2012, www.usenix.org/system/files/conference/hotdep12/hotdep12-final17.pdf
-

